

B.A.T. WING

3DPAC Rotary Wing Autorotation Aircraft Final Technical Report

FIRST PLACE ROTARY WING CATEGORY

First successful autorotation-based drag device demonstrated in the competition

A final report submitted to:

Dr. Robert Woods and Dr. Thomas Allsup
The University of Texas at Arlington

Faculty Advisors:

Dr. Robert Taylor and Dr. Thomas Allsup

Team Members:

Zachary Precourt | Ryan Hailey | Diego De Larranaga | Nicholas Rios | Ramez Sager

Date: 8/6/2026

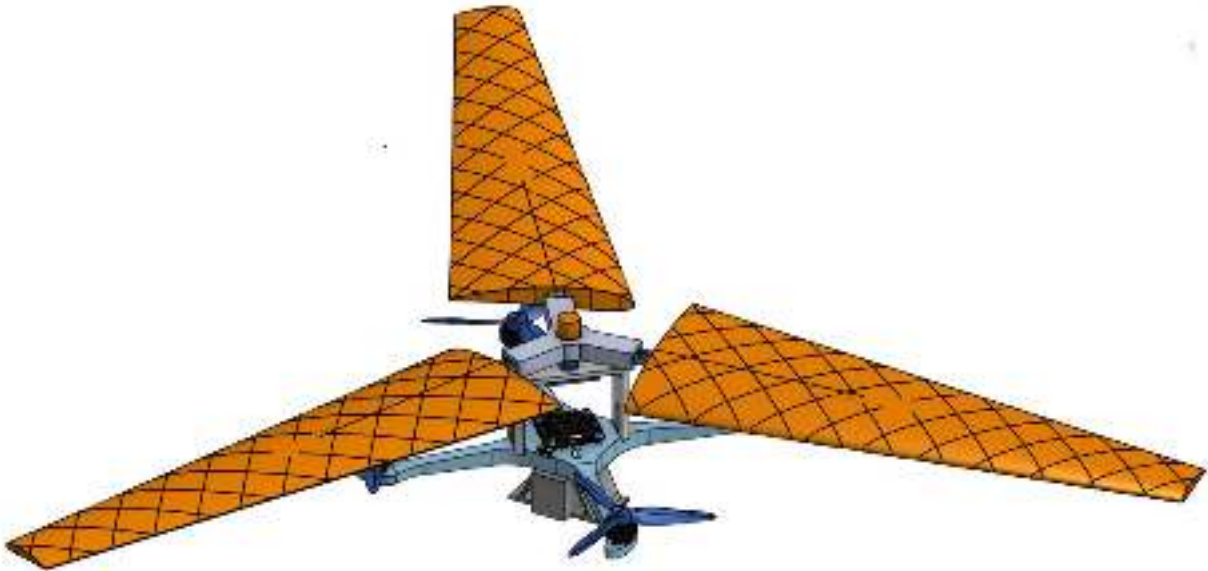


Table of Contents

Contents

Table of Contents	2
0. ABSTRACT	5
0.1 Project Personnel and Technical Ownership	5
0.2 Faculty and Facility Support	5
0.3. NOMENCLATURE	6
1. INTRODUCTION, BACKGROUND, AND MOTIVATION	7
1.1 Competition Background	7
1.2 Project Motivation	7
1.3 Project Objective	7
1.4 Scope of This Report	7
2. DESIGN REQUIREMENTS AND CONSTRAINTS	8
2.1 Competition Requirements	8
2.2 Team Level Design Targets	8
2.3 Functional Success Criteria	8
2.4 Safety and Regulatory Constraints	9
3. CONCEPT SELECTION AND DECISION MATRIX	10
3.1 Preliminary Concept Development	10
3.2 Weighted Decision Matrix	10
3.3 Score Justification	10
3.4 Final Concept Selection	10
4. FINAL DESIGN OVERVIEW	11
4.1 System Design	11
4.2 Mechanical Layout and Design Rationale	11
4.3 Design Phase Geometry	11
5. AUTOROTATION THEORY	12
5.1 Physical Description	12
5.2 Coupled Vertical and Rotational Dynamics	12
5.3 Blade Element Momentum Theory	12
5.4 Conditions Required for Successful Autorotation	13
6. AERODYNAMIC OPTIMIZATION AND ANALYSIS	14
6.1 Optimization Objective and Variables	14
6.2 Software Structure	14
6.3 Airfoil Screening	14
6.4 Selected Rotor Geometry	14
6.5 Predicted Performance	14
6.6 Interpretation of the Analysis	14

7. MANUFACTURING AND MECHANICAL INTEGRATION.....	16
7.1 Material Selection	16
7.2 Blade Design for Printing.....	16
7.3 Manufacturing and Assembly Sequence	17
7.4 Assembly Quality Control and Ground Testing	18
8. ELECTRONICS, PROPULSION, AND SEMI-AUTONOMOUS OPERATION.....	19
8.1 Propulsion Sizing	19
8.2 Design Phase Hardware Baseline.....	19
8.3 Hardware Evolution and Final Competition Configuration	20
8.4 Semi-Autonomous Control Design	20
Why Autonomy?	20
Electronics.....	20
Software for Autonomy.....	21
Control Structure and Implementation	22
Cascaded Altitude Velocity Controller	23
Web UI	34
9. TESTING, TROUBLESHOOTING, AND ITERATION.....	38
9.1 Test Philosophy	38
9.2 Recommended Test Progression	38
9.3 Electrical and Software Troubleshooting	38
9.4 Mechanical Troubleshooting.....	38
9.5 Data Collection.....	39
10. RESULTS AND COMPETITION PERFORMANCE	39
10.1 Design Phase Predictions	39
10.2 Observed Physical Performance.....	39
10.3 Competition Result.....	39
10.4 Comparison with the Numerical Model	40
10.5 Reliability and Repeatability	40
11. SUMMARY AND CONCLUSIONS.....	41
12. FUTURE WORK	42
12.1 Electronic Improvements	42
12.2 Airframe and Manufacturing Improvements.....	42
12.3 Control and Autonomy Improvements.....	42
13. ACKNOWLEDGEMENTS	43
14. REFERENCES.....	44
APPENDIX A. COMPETITION RULES, CODES, AND COMPLIANCE	45
APPENDIX B. DECISION MATRIX SCORE JUSTIFICATION	46
APPENDIX C. OPTIMIZATION ASSUMPTIONS	46
APPENDIX D. OPTIMIZATION AND SOFTWARE GITHUB REPO	48

APPENDIX E. PROPULSION CALCULATIONS AND COMPONENT DATA	49
--	----

0. ABSTRACT

This report documents the complete development of the B.A.T. Wing, a lightweight, additively manufactured aircraft created for the University of Texas at Arlington 3D Printed Aircraft Competition (3DPAC) Rotary Wing category. The work progressed from preliminary sketches and formal concept selection through aerodynamic modeling, rotor optimization, CAD, additive manufacturing, electronics integration, Betaflight configuration, and flight testing. The final mission structure separated the flight into a short quadcopter style powered ascent and a passive autorotating descent.

A weighted decision matrix compared an autorotating rotor aircraft, a VTOL tail sitter glider, and a glider quadcopter. Autorotation was selected because it produced useful lift and aerodynamic braking after motor cutoff without additional battery energy, avoided a difficult transition to forward flight, and fit the limited competition field. The descent system used a three blade rotor. Blade element momentum theory and coupled vertical/rotational dynamics were implemented in Python using AeroSandbox, NeuralFoil, and CasADi. The design phase optimization selected a NACA 4512 airfoil with a 343.30 mm rotor radius, 99.98 mm root chord, 33.34 mm tip chord, 6.56 deg root twist, and 0.55 deg tip twist.

The aircraft was manufactured using ASA Aero for lightweight blade shells, ASA for the primary body, and Onyx carbon fiber reinforced nylon for spars. A four-motor propulsion system, flight controller, ESC, and discharge 3S battery was integrated and configured in Betaflight. Betaflight supplied high-rate attitude stabilization and motor mixing while the pilot provided arming and throttle. This operating method is described as semi-autonomous because the controller automatically stabilized the aircraft, but the mission was not a fully programmed waypoint flight.

Progressive tests were used to reduce risk. Before performing low level flight tests, part inspection, continuity/voltage checks, sensor calibration, individual motor tests with propellers removed, and rotor balance were all checked. The project earned first place in the Rotary Wing category and became the first team in the competition to successfully demonstrate an autorotation-based drag device. The competition result validates the overall mission strategy and integration approach.

0.1 Project Personnel and Technical Ownership

Table 0.1. Project personnel, responsibility areas, and handoff records.

Name	Primary Responsibility Area	Contact
Zachary Precourt	Team lead / project management	zmp0920@mavs.uta.edu
Ryan Hailey	Optimization / electronics	rmh6127@mavs.uta.edu
Ramez Sager	Documentation / electronics / testing	ros8619@mavs.uta.edu
Nicholas Rios	Manufacturing, materials, and testing	nrx0043@mavs.uta.edu
Diego De Larranaga	CAD / propulsion integration / autorotation	dxd3313@mavs.uta.edu

0.2 Faculty and Facility Support

Dr. Robert Taylor and Dr. Thomas Allsup provided technical guidance, design feedback, and competition support. Steven McDermott provided access to laboratory space for assembly, soldering, electronics troubleshooting, bench testing, and competition preparation.

0.3. NOMENCLATURE

Table 0.2. Nomenclature used in the report.

Symbol / Acronym	Description
FC	Flight controller
ESC	Electronic speed controller
BEC	Battery eliminator circuit
UART	Universal asynchronous receiver transmitter serial port
PWM	Pulse width modulation
BEM	Blade element momentum theory

1. INTRODUCTION, BACKGROUND, AND MOTIVATION

1.1 Competition Background

The 3D Printed Aircraft Competition (3DPAC) challenges student teams to design, manufacture, and operate aircraft that use additive manufacturing as a central part of the airframe. The B.A.T. Wing was entered into the Rotary Wing category, where a significant portion of the useful aerodynamic force is produced by rotating components. The competition mission limited the duration of powered flight, restricted altitude, restricted field boundaries, required safe operation, and recorded the best result from a limited number of attempts.

These rules made the mission fundamentally different from normal endurance flight. A large battery and continuous motor operation could not directly produce the winning result. The vehicle instead needed to gain useful altitude quickly, stop powered lift before the time limit, and convert the remaining gravitational potential energy into a slow and controlled descent.

1.2 Project Motivation

The team focused on the unpowered portion of the mission because it represented the largest opportunity to increase total time aloft. A conventional multirotor would lose nearly all lifts when the motors stopped. A fixed wing or tail sitter concept could glide, but it would need adequate altitude, air speed, authority control, and horizontal room to complete the transition from vertical climb to efficient forward flight. The limited field and short mission increased the risk associated with that transition.

Autorotation offered a passive alternative. During descent, upward relative airflow through the rotor disk can drive blade rotation. The rotating blades can then produce lift and aerodynamic braking without additional battery power. This approach directly matched the scoring objective and allowed the aircraft to remain vertical rather than requiring a long glide path.

1.3 Project Objective

The project objective was to create a lightweight, 3D printed rotary wing aircraft that could complete the powered ascent requirement and maximize time aloft during an unpowered autorotating descent. The aircraft also had to remain practical to manufacture, integrate, repair, and operate.

1.4 Scope of This Report

This report documents the complete engineering process rather than only the final vehicle. It presents the requirements, concept selection, theory, numerical analysis, CAD, manufacturing decisions, propulsion sizing, electronics, Betaflight configuration, semi-autonomous control structure, test progression, troubleshooting, competition performance, and future team recommendations.

2. DESIGN REQUIREMENTS AND CONSTRAINTS

2.1 Competition Requirements

Table 2.1. Competition requirements and B.A.T. Wing design responses.

Requirement	Design Response
Rotary Wing category	Use a three-blade autorotation rotor as the primary passive descent device.
Maximum 8s continuous powered flight	Size the propulsion system for rapid ascent rather than endurance.
Altitude below 60ft	Use a mostly vertical trajectory and establish a reliable method to remain below the ceiling.
Approximate field size	Avoid concepts that need a long glide transition or large horizontal travel.
3D printed aerodynamic surfaces	Design blade shells, body, and structural aerodynamic elements around printable geometry and available materials.
Safe electric propulsion; no lighter than air or unsafe methods	Use conventional brushless motors, propellers, battery, flight controller, and ESC.
Best of three attempts recorded	Prioritize repeatability, repairability, spare parts, and quick preflight/post flight inspection.

2.2 Team Level Design Targets

Table 2.2. Design phase targets and required final verification.

Parameter	Design Phase Value	Final Competition Value
Vehicle mass	400 g nominal analysis value; 500 g planning limit	300 g
Overall span	Approximately 749 mm	Same
Overall height	Approximately 110 mm	Same
Autorotation blades	Three	Three
Powered lift motors	Four	Four
Powered duration	Maximum 8 continuous seconds	Same
Altitude ceiling	Below 60ft	Same (excluding overshoot)
Thrust to weight target	Two	Three

2.3 Functional Success Criteria

The project was considered successful only if the complete system worked as a mission, not merely if individual parts functioned. The practical criteria were controllable powered ascent, reliable motor cutoff, rotor spin up after cutoff, measurable slowing of the descent, safe landing or repairable impact, repeatability across attempts, and compliance with the competition rules.

2.4 Safety and Regulatory Constraints

Bench and flight work required propeller removal during configuration, monitored power up, continuity checks before connecting a battery, secure battery retention, field boundary awareness, and a clear abort procedure.

3. CONCEPT SELECTION AND DECISION MATRIX

3.1 Preliminary Concept Development

The team began with preliminary sketches and early CAD rather than immediately committing themselves to one design. Three concepts were retained for formal comparison: an autorotating rotor aircraft with powered multirotor ascent, a VTOL tail sitter glider, and a glider quadcopter hybrid. Early drawings were used to identify major interfaces.

3.2 Weighted Decision Matrix

The matrix emphasized hang time and weight because those factors most directly affected the score and mission feasibility. Fabrication, powertrain complexity, integration complexity, and development effort were included because a theoretically strong concept would still fail if the team could not manufacture and test it within the available time. Higher raw scores represented more favorable expected performance.

Table 3.1. Concept selection decision matrix.

Criterion	Weight	Autorotation	Tail-Sitter	Glider/Quad Hybrid
Hang time	35%	8	9	5
Weight	25%	8	5	6
Fabrication	15%	7	5	7
Powertrain complexity	10%	8	9	5
Integration complexity	10%	7	4	3
Development effort	5%	8	4	4
Weighted total	100%	7.75	6.65	5.30

3.3 Score Justification

The tail sitter received the highest raw hang time score because a successful glider transition could be efficient, but it was penalized for mass, fabrication, integration, and development risk. The glider quadcopter hybrid was easier to visualize but required more structure, a clean transition, and control of multiple flight regimes. The autorotation concept received a slightly lower raw hang time score than the tail sitter, but it provided the strongest overall balance and avoided the need for a long horizontal flight path.

3.4 Final Concept Selection

The autorotation concept achieved the highest weighted total and was selected. The choice was significant because it separated the mission into two specialized systems: a compact powered system for ascent and a passive rotor for descent.

4. FINAL DESIGN OVERVIEW

4.1 System Design

The B.A.T. Wing combined a lower four motor frame with an upper three blade free spinning rotor. The lower structure carried the motors, propellers, battery, flight controller, ESC, receiver, and wiring. The upper structure carried the rotor mast, hub system, spars, blade roots, and lightweight airfoil shells.

4.2 Mechanical Layout and Design Rationale

Motor thrust axes were arranged symmetrically around the vehicle center. The battery and electronics were placed near the center to reduce unwanted pitch and roll moments. The rotor mast and hub were designed to remain stiff while allowing the rotor to spin with minimal friction. Blade attachment had to be repeatable because small differences in pitch, alignment, or mass could produce rotor wobble and reduce autorotation efficiency.

4.3 Design Phase Geometry

Table 4.1. Design phase rotor and vehicle geometry.

Parameter	Value
Airfoil	NACA 4512
Number of blades	3
Rotor radius	343.30 mm
Root chord	99.98 mm
Tip chord	33.34 mm
Root twist	6.56 deg
Tip twist	0.55 deg
Overall span	Approximately 749 mm
Overall height	Approximately 110 mm

5. AUTOROTATION THEORY

5.1 Physical Description

Autorotation occurs when airflow through the rotor drives rotation without motor torque. As the B.A.T. Wing descends, the relative airflow passes upward through the rotor disk. Depending on the local blade pitch and inflow angle, some blade regions produce positive driving torque while the complete rotor produces an upward aerodynamic force. Falling vehicles supply the energy.

5.2 Coupled Vertical and Rotational Dynamics

The performance of an autorotation device is primarily governed by two equations. Vertical dynamics of the rotor and payload as well as the rotational dynamics of the rotor itself. The vertical motion is modeled by balancing the acceleration of gravity with the upward force provided by the rotor thrust and drag force from vertical descent:

$$a = g - \frac{T + F_{\text{drag}}}{m_{\text{total}}} \quad (1)$$

The vertical fall dynamics are linked to the rotational dynamics of the rotor. The rotational velocity ω directly influences the vertical acceleration of the platform, and the vertical velocity of the platform influences the angular acceleration of the rotor:

$$\frac{d\omega}{dt} = \frac{Q_{\text{aero}} - k\omega|\omega|}{I_{\text{rotor}}} \quad (2)$$

Accounting for all of the possible losses experienced by the 3d printed assembly is outside of the scope of this project's development. We approximated the additional losses as a function of a tunable k parameter multiplying the square of the rotor's angular velocity.

5.3 Blade Element Momentum Theory

It is computationally intractable to solve a continuous analytical solution for the rotor dynamics across the entire span of the blade. A common way of overcoming this limitation is using the Blade Element Momentum method, which discretizes the blade into a finite number of sections. This allows us to approximate the cross section of the blade as relatively constant for each of the finite span sections. In doing this we can also approximate the airflow over the section as exactly in-plane with the 2D cross-section. The integration process of the net effect is simple from here, as the only thing left to do is to sum up the net effect of each blade section across the entire rotor

The blade is split into N_{span} radial stations.

At each station:

- Local relative wind is computed from descent speed, rotor speed, and induced flow
- Inflow angle: $\phi = \arctan \frac{v_a}{v_t}$
- Angle of attack: $\alpha = \phi - \beta(r)$
- Airfoil polar lookup gives C_L and C_D
- Local thrust and torque are computed
- Contributions are summed spanwise with Prandtl tip and hub loss corrections

Using this, we can get a computationally feasible model of the performance of each autorotation blade candidate.

5.4 Conditions Required for Successful Autorotation

A blade that has an airfoil shape will not automatically autorotate. Successful operation depends on the pitch and twist distribution, blade area, rotor inertia, Reynolds number, hub and bearing friction, blade mass matching, structural stiffness, vehicle mass, and initial spin up conditions. Excessive friction or poor pitch distribution can prevent spin up. Excessive blade mass can increase inertia and slow acceleration, while insufficient inertia can allow speed to fluctuate.

6. AERODYNAMIC OPTIMIZATION AND ANALYSIS

6.1 Optimization Objective and Variables

Python was used to evaluate the design. The optimization sought a rotor geometry and descent trajectory that increased fall time while respecting geometric, mass, aerodynamic, and landing constraints. The design variables included rotor radius, root chord, tip chord, root twist, tip twist, collective pitch, final time, and time histories of height, descent velocity, and rotor angular velocity.

6.2 Software Structure

Table 6.1. Software tools used in the aerodynamic and structural workflow.

Tool	Purpose
AeroSandbox	Organized geometry, aerodynamic calculations, and optimization variables.
NeuralFoil	Estimated differentiable low-Reynolds-number airfoil lift and drag behavior [3].
CasADi	Constructed symbolic dynamic constraints and derivatives for nonlinear optimization [6].

6.3 Airfoil Screening

Several airfoil profiles were considered during the aerodynamic design process to determine which shape would be most suitable for the autorotation blades. The airfoils were compared using numerical analysis based on their predicted lift and drag characteristics under the expected operating conditions. The purpose of this screening was to narrow the design space before performing the more detailed rotor optimization. Rather than selecting an airfoil based on a single aerodynamic value, the team considered overall predicted performance and how well each profile worked within the complete rotor design. Based on this process, the NACA 4512 airfoil was selected for the final autorotation blades and was then used in the detailed rotor geometry optimization.

6.4 Selected Rotor Geometry

The design phase optimization selected NACA 4512 with the geometry listed in Table 4.1. The selected profile and taper/twist distribution represented the best compromise among predicted descent time, landing velocity, rotor mass, manufacturability, and solver feasibility. The optimization result served as the starting point for CAD and printing rather than as an exact prediction of competition performance.

6.5 Predicted Performance

The design phase simulation predicted that the powered system could meet the 60ft ascent requirement within the allowed powered duration. For the unpowered phase, the model predicted rotor spin up, a reduction in sink rate through combined lift and drag, and an estimated descent duration of approximately 13s from the modeled release height. The predicted curves included height, descent speed, and rotor RPM.

6.6 Interpretation of the Analysis

The aerodynamic analysis showed that the autorotation concept was capable of producing useful lift and aerodynamic braking during the unpowered descent phase. The optimization process indicated that rotor geometry, particularly blade radius, chord, twist, and airfoil selection, had a significant effect on rotor speed and descent performance. Based on these results, the NACA 4512 airfoil and optimized blade geometry were selected as a balance between aerodynamic performance, low weight, and manufacturability. The analysis also showed that increasing rotor area could improve descent performance, but larger blades introduced additional weight, drag during powered ascent, and structural concerns. Therefore, the final rotor was not selected based only on maximum theoretical lift. Instead, the geometry represented a compromise between autorotation performance and the practical limitations of the complete aircraft.

Overall, the simulation results supported continuing with the three blade autorotation design and provided the dimensions used to develop the final CAD model. The analysis predicted that the rotor would be capable of sustaining rotation during descent and reducing the aircraft's sink rate without additional motor power. Physical

testing was then used to determine how closely the manufactured aircraft matched these predictions and to identify effects that were not fully represented in the analytical model.

7. MANUFACTURING AND MECHANICAL INTEGRATION

7.1 Material Selection

Material selection was based on the structural, aerodynamic, and manufacturing requirements of each component rather than using a single filament throughout the vehicle. Because the autorotation blades represented a large portion of the vehicle's projected area, minimizing their mass was especially important for reducing disk loading and improving descent performance. In contrast, the central body, attachment points, and internal spars required greater stiffness, dimensional stability, and resistance to handling loads. The final material combination therefore balanced low density, structural performance, print reliability, cost, and compatibility with the available manufacturing equipment. Of the selected materials, ASA Aero imposed the greatest design and manufacturing constraints because its foaming behavior required the blade geometry and slicer strategy to be developed concurrently.

Table 7.1. Additive manufacturing materials and intended use.

Material	Primary Use	Reason for Selection
ASA Aero	Autorotation blade shells / airfoils	Foaming behavior reduces effective density and enables large lightweight surfaces.
ASA	Main body and general structural components	Durability, weather resistance, temperature resistance, and practical outdoor use.
Onyx carbon fiber reinforced nylon	Internal spars / stiffening members	Higher stiffness and strength than standard thermoplastics in critical structural regions.

7.2 Blade Design for Printing

ASA Aero is a foaming material, which creates both an advantage and a manufacturing challenge. The team used a continuous path strategy, known as vase mode, to limit stops and starts that could cause stringing, clogging, or inconsistent expansion. The blade geometry therefore had to be designed so that each layer could be produced as a continuous loop. Achieving this required a non intuitive modeling approach that used Boolean operations to manipulate how the slicer interpreted the part. Rather than modeling only the material that would remain in the finished blade, solid bodies were also extruded through regions intended to be hollow and then combined or subtracted in a specific sequence. This effectively guided the slicer into identifying a continuous outer perimeter at each layer while preserving the desired internal cavities. Internal ribs and spar cutouts were incorporated into the geometry to improve stiffness without turning the blade into a solid, heavy part. Figure 7.1 illustrates the sequence of Boolean operations used to create the internal blade cavities while maintaining a continuous exterior perimeter that could be interpreted by the slicer.

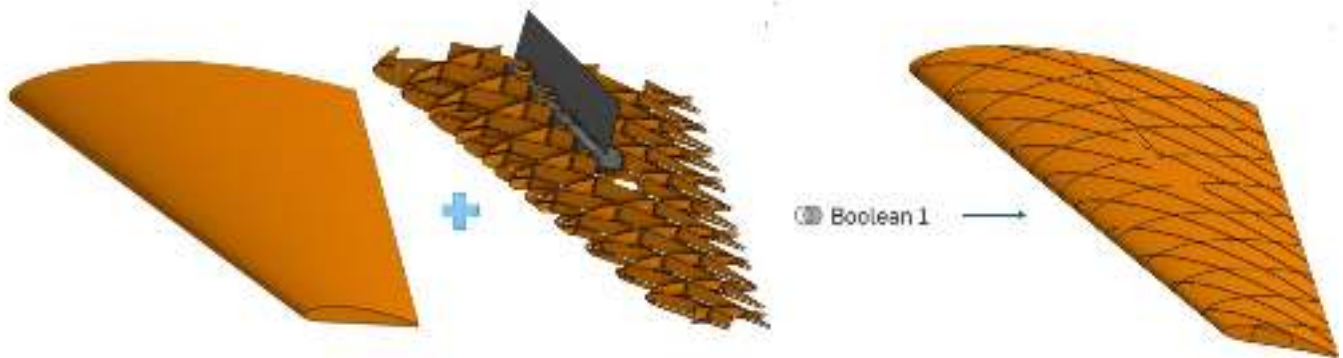


Figure 7.1 Boolean Operation

7.3 Manufacturing and Assembly Sequence

Strategically scheduling the manufacture and assembly of individual components helped prioritize mission critical hardware while preserving sufficient material and replacement parts for ground and flight testing. The production strategy focused first on components with the greatest manufacturing complexity, longest lead times, and highest likelihood of failure or rework.

The ASA Aero blades represented the primary manufacturing bottleneck because of their complex geometry, sensitivity to print settings, and specialized material behavior. The main chassis represented the next most significant production requirement; however, its conventional ASA construction allowed multiple units to be produced reliably once the design was finalized. Smaller secondary components required less material and shorter print times and were therefore manufactured after the blade and chassis production processes had been established. The detailed manufacturing and assembly sequence was as follows:

1. Print small ASA Aero material coupons and short airfoil sections to validate the blade manufacturing process before committing to full scale production.
2. Refine and confirm the nozzle temperature, material flow, wall thickness, print speed, dimensional expansion, and slicer settings required to produce consistent blade geometry.
3. Begin full scale blade production using a standardized orientation, material batch, printer, and slicer profile. Prioritize printing additional blades to provide replacements for destructive testing, flight testing, or manufacturing defects.
4. Inspect each blade for warping, inconsistent expansion, surface defects, and dimensional accuracy, with particular attention given to the leading edge, trailing edge, spar channel, and root interface.
5. Begin large scale production of the main chassis once its geometry is finalized. Produce multiple chassis components as needed to support assembly trials, structural testing, and potential replacement following flight tests.
6. Manufacture the smaller secondary components, including the hub, motor mounts, electronic mounts, and other connection hardware, after the primary blade and chassis production processes are established.
7. Install the blade spars and dry fit the blades, hub, chassis, and secondary components to verify alignment, clearances, and proper interface geometry before applying adhesive or permanent fasteners.
8. Assemble the motors, electronics, battery, and remaining hardware onto the chassis while continuously monitoring the vehicle's total mass and center of gravity location.
9. Conduct initial system checks and ground testing, then repair or replace components as necessary using the additional parts produced earlier in the manufacturing process.
10. Perform a final inspection of blade clearances, structural joints, electrical connections, fasteners, and rotating components before applying full power or beginning flight testing.

7.4 Assembly Quality Control and Ground Testing

Quality control activities focused primarily on verifying that the manufactured components could be assembled correctly and function as intended before flight testing. Because several components were produced independently and later joined into a single vehicle, fit checks were performed throughout assembly to identify interference, misalignment, insufficient clearances, and inconsistencies between printed interfaces.

Before permanent fastening, the blades, spars, hub, chassis, motor mounts, and secondary components were dry fitted to confirm proper alignment and assembly. Particular attention was given to the blade root and hub interfaces, spar placement, motor clearances, and the available space for electronics and wiring. Components that did not fit as intended were modified or reprinted before proceeding with final assembly. This reduced the likelihood that dimensional inconsistencies or accumulated manufacturing tolerances would affect the completed vehicle.

Following assembly, ground and rotational testing were conducted to evaluate the operation of the propulsion and autorotation systems under controlled conditions. These tests were used to identify interference between rotating and stationary components, excessive vibration, loose fasteners, abnormal structural motion, and other behavior that could indicate an assembly or manufacturing defect. Electrical connections, motor operation, blade clearance, and structural attachment points were checked before and after each test.

The results of these checks were used to guide component replacement, fastening adjustments, and minor geometric revisions before flight. Although the quality control process was primarily intended to verify assembly fit and function, it also helped improve structural reliability, maintain adequate rotating clearances, and reduce the risk of preventable failures during flight testing.

8. ELECTRONICS, PROPULSION, AND SEMI-AUTONOMOUS OPERATION

8.1 Propulsion Sizing

The propulsion system was sized for the short climb rather than continuous endurance. The mission envelope rewarded marginal performance, so the team sought to select the smallest battery and motors possible. To select hardware, it was first necessary to define the thrust to weight ratio (TWR) and target weight requirements for the drone. These were defined in previous sections. The thrust to weight ratio was based on typical hobbyist drones, which typically range in anywhere from 3-8 TWR. Given our marginal performance goal, the team felt comfortable targetting a TWR of 2. This surpasses TWRs used in other vehicles such as rockets, as the drone's mass properties necessitate a larger control authority to disturbances, but remains far below that of COTS drones.

Our target weight was obtained from similar research on drone weight categories, which revealed an inverse relationship between TWR and vehicle size. Informed by early chassis models, the smallest drone category capable of withstanding outdoor disturbances was selected: the 200–400 gram microdrone category. With a TWR and target goal defined, motors that met the derived thrust requirement could be selected. Empirical thrust data for motor propeller pairs was obtained from Recursion Labs [19] and validated by manufacturer specifications. Once a motor was selected with a corresponding propeller size to capitalize on its performance, the amperage requirements could be calculated to determine the minimum viable battery performance. The battery was selected based on its ability to sustain 3 flights without voltage sag, so the rough energy capacity calculation was based on double the required energy for all flights. The result was a low capacity and high discharge battery. These electronics informed the selection of the ESC and flight controller to sustain the amperage requirements.

8.2 Design Phase Hardware Baseline

Table 8.1. Design phase propulsion and electronics baseline.

Component	Qty	Unit Weight (g)	Total (g)
Happymodel EX1404 3500KV motors	4	9	36
Gemfan 4 inch propellers	4	1.5	6
450mAh 3S 80C LiHV	1	29	41
BetaFPV F4 2-3s AIO	1	6	6
TOF sensor	1	3	3
ESP32-S3 Supermini	1	3	3
Wires	10	1	10
Screws	10	1	5
Hardware total	-	-	110

8.3 Hardware Evolution and Final Competition Configuration

For the final configuration that was implemented for the competition can be seen in the following figure below.

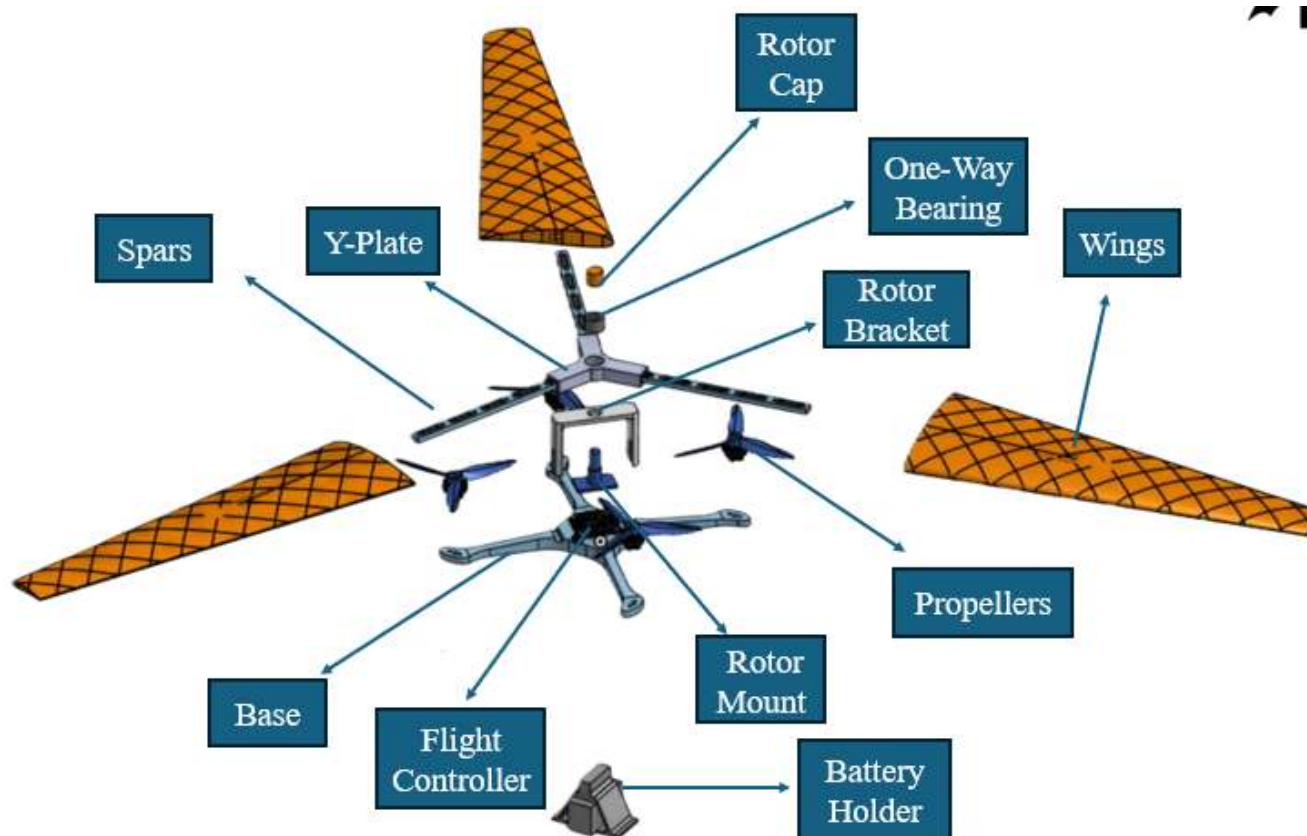


Figure 8.3. Final Drone Configuration

8.4 Semi-Autonomous Control Design

The B.A.T. Wing was semi-autonomous rather than fully autonomous. The pilot provided high level commands, while Betaflight closed the fast stabilization loops. The flight controller continuously read the gyroscope and accelerometer, compared measured motion with commanded attitude or rates, calculated roll/pitch/yaw corrections using the PID controller, mixed those corrections into four motor commands, and sent the commands to the ESC. The pilot remained responsible for arming, climb throttle, flight path corrections, the command to end powered flight, and abort decisions.

Why Autonomy?

For the scope of this competition, which involves flying up to a predetermined altitude within a fixed time allotment in a highly repeatable fashion, there is no need for a human pilot. Having a deterministic programmable flight logic run onboard (semiautonomy) allows your drone to maximize the powered flight regime. It also allows your drone to reliably take advantage of a loophole in the rules: throttling up toward the end of the eight second flight to overshoot the altitude ceiling (staying within the 3 second allotment). There is another benefit in that there is an “objective” truth from a judging standpoint. You will have the ability to directly pull telemetry over the air off the drone after each flight, so the likelihood of your team winning contested challenges against a successful run goes up as well.

Electronics

Based on our previously discussed power requirements for flying our quadcopter, a 3S LiPo battery was chosen, so our flight controller (FC) configuration needed to be 3S compatible. The team was fortunate enough to source a board optimized for 2–3S battery operating voltages, the BetaFPV F4 2–3S AIO. We originally chose a SpeedyBee

F405 Mini stack. This is a highly capable board that accepts a large range of input battery voltages because of its more robust, separate ESC board, which can handle a greater electrical load. The BetaFPV F4 2–3S AIO greatly reduced the complexity of the ESC portion of the flight controller, resulting in a 50% reduction in weight compared with the SpeedyBee F405 Mini.

The BetaFPV F4 exposes three UART TX–RX pairs for the operator to use. The chosen pads were T6 and R6 for bidirectional communication with an ESP32-S3 Super Mini using the MultiWii Serial Protocol (MSP). The FC’s T6 transmit pad connects to the ESP32’s receive pin, while the FC’s R6 receive pad connects to the ESP32’s transmit pin. Through MSP, the ESP32 requests and receives flight data such as altitude, vertical velocity, attitude, arming status, and receiver channel status. In the opposite direction, the ESP32 transmits MSP_SET_RAW_RC messages containing throttle, yaw, arm, and flight-mode channel values. These messages allow the ESP32 to emulate, or “fake,” the human RC commands that Betaflight normally receives from a radio receiver. The ESP32 also provides Bluetooth based configuration and mission commands through the browser interface; however, the time critical control and telemetry exchange between the ESP32 and flight controller occurs over the wired MSP UART connection.

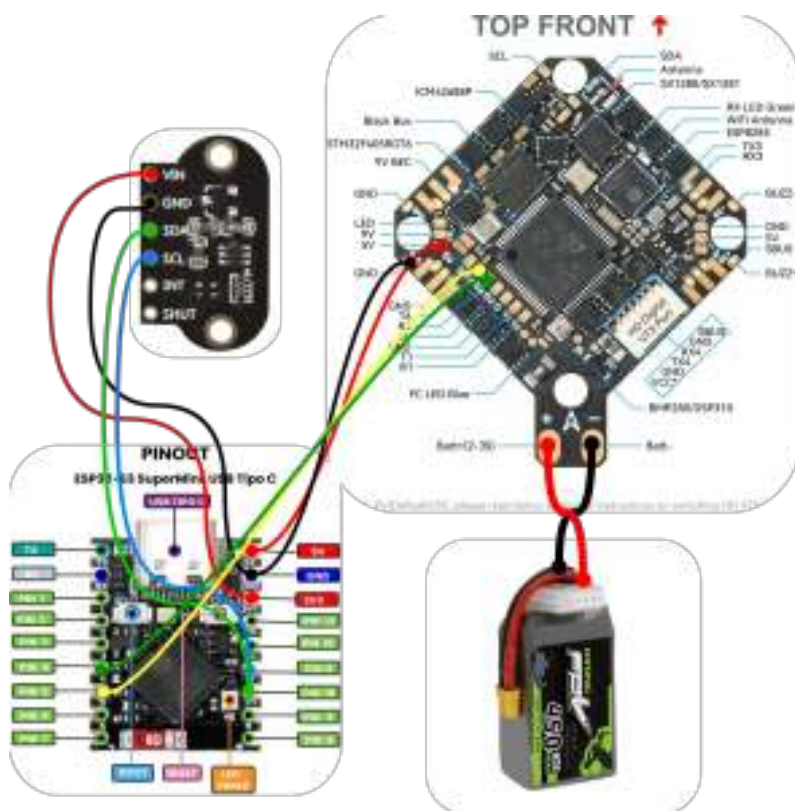


Figure 8.1. Electronics Wiring Diagram.

An add a time-of-flight (ToF) sensor was also chosen for better low altitude performance. We want to emphasize that a ToF sensor is NOT necessary for this competition. Refer to the *Mistakes and Recommendations* section for further elaboration. The ToF can be useful for landing scenarios, but that is not a requirement for competition, so we recommend ToF sensors to be ignored unless all other requirements have already been met and validated.

Software for Autonomy

The mission requirement for the 3DPAC competition is straight forward. No obstacles avoidance, no target tracking, no waypoints, etc.... For that reason, the software stack required to achieve autonomous flight is also straightforward. BetaFlight is a well validated and easy to use low level quadcopter flight firmware that has a large base of compatible flight controllers. More than likely if you have a commercially available flight controller, there is also a corresponding build of BetaFlight ready to run on your board. In the case of Team Batwing, our FC board

came with a pre-configured version of BetaFlight. Specifically, version 4.4.2 with the BetaFPV F405 target, made for the Pavo Pico racing drone model.

The team had to reflash the FC with a fresh version of BetaFlight 4.4.3 to get some features working properly. More elaboration of the struggles with the pre-configured flight firmware are detailed in *Mistakes and Recommendations*. Once the team had a properly functioning BetaFlight build flashed onto the FC, the development could start the higher level controller on the ESP32-S3 supermini. Thankfully Espressif maintains a robust ecosystem compatible with the Arduino framework, so the team could leverage plenty of existing libraries to make our development process faster.

Even though Arduino was used for the framework to write, compile, and flash code onto the ESP32, we do not recommend using Arduino IDE to further develop our repository or develop your own. Arduino CLI used in VS Code was the better choice, and the path we recommend for future teams. It will allow the user to easily develop the project across many different files and easily compile specific .ino sketches with simple CLI commands, meaning the user can have multiple builds and sketches inside the same repository, something that will feel very limiting in Arduino IDE if you decide to pursue that route.

The team only required two libraries from Arduino and two other 3rd party libraries compatible with the Arduino framework. It should be noted that third party libraries must be installed separately before compiling.

Table 8.2. Library List for C++ development.

Header	Category	Purpose
Arduino.h	Arduino framework	Provides setup (), loop (), Serial, millis (), micros (), pinMode (), delay Microseconds (), GPIO APIs, and other Arduino functions
Wire.h	Arduino core library	Provides I ² C communication for sensors and peripherals
NimBLEDevice.h	Third party Arduino library	Provides Bluetooth Low Energy support using NimBLE
VL53L1X.h	Third party Arduino library	Provides a driver for the VL53L1X time-of-flight distance sensor

The team also developed a web interface deployed to GitHub pages for access to the ESP32 tunable parameters and mission start UI from any device that has internet connection and a web browser. The web UI as developed using HTML, CSS, and JS inside the same repository as the Arduino C++.

Control Structure and Implementation

To summarize the mission phases discussed earlier, Arm → Sprint → Hold → Punch → Cut Power, it can be seen that the team had four finite states. As a result, the control architecture also implements a state machine for the competition mission.

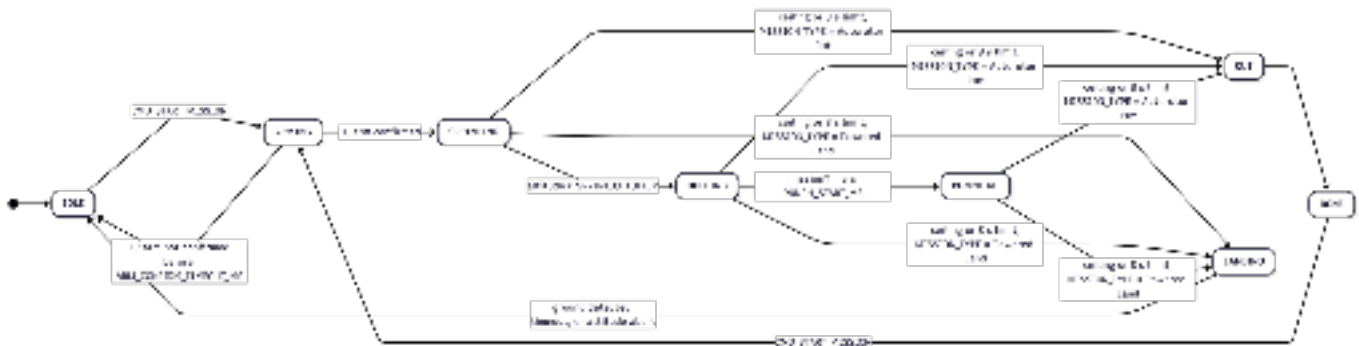


Figure 8.2. Mission State Machine.

It can be noticed that some gates were added and are checked on top of the basic four states (arm, sprint, hold, punch). These were implemented as patterns of issues were seen in different mission attempts. For example, the team had an arming loop that pulls the drones defined aux arm channel to the correct throttle value in a loop until BetaFlight arms and sends a confirmation, or our try loop times out, and it goes back to idle state for inspection.

Once the FC has been armed and sends a confirmation back, the state progresses to the sprinting phase. In the sprint phase, the goal is to get to the upper end of the altitude cap as fast as possible. As a result, we hard code maximum throttle in this state. No altitude control loop involved at all in SPRINTING state. There are three ways to exit SPRINTING. One being a successful ascent up to the allowable fused altitude reading `SPRINT_CUTOFF_M` to start using the cascaded controller to get up to the ultimate goal altitude of 18.8 meters (60 feet) in the HOLDING state. The second exit case is an unsuccessful ascent up to the desired altitude in time and having the hard coded eight second timer either kill the motors, or go into LANDING state depending on the `MISSION_TYPE` (we have a Powered Land mission type that allows us to test the drone without breaking it too badly. Instead of it falling to the ground for each altitude test, we can fly up to altitude and see what our mission flight profile looks like before it falls, then gradually descend). If it is the true competition mission (Auto rotor Cut) then after 8 seconds no matter what, the ESP32 will disarm the motors, and the drone will fall unpowered. Once state has transitioned to HOLDING, then we have a cascaded controller altitude velocity controller (Fig. 8.3) take over the rest of the way until the PUNCHING state is transitioned to. Punching for our drone involves a final kick of vertical throttle and maximum clockwise yaw command. This allows us to start our “descent” with positive vertical velocity as well as pre spin out autorotation device, which throughout testing may be the most important feature for our drone’s performance. Our one way bearing fitting for the hub of the auto rotor allowed us to be able to spin the rotor by spinning the drone itself, eliminating the need to carry extra actuators and mounting hardware. During the SPRINTING state, there can be an early exit if maximum altitude ceiling is reached (22 meters) or eight seconds of powered flight is exceeded. Depending on mission type, the motors will either be fully disarmed or go into a landing sequence after PUNCHING is complete.

Cascaded Altitude Velocity Controller

We mentioned an implementation of a cascaded controller in the previous section without expanding upon it. This section will cover the finer details. We will first start with the goal of the controller:

To minimize the error between pilot or mission defined commanded altitude and the drone’s current altitude. We initially tried to do this naively with a simple altitude PID controller, only controlling altitude position error. It turns out the delta from 0 to 18.8 meters was too much error for the controller to handle reasonably, and the small drone was way too responsive to a pure altitude control loop alone. A slightly more sophisticated implementation of the PID involved nesting controller loops, otherwise known as cascaded control. The outer loop being a slower altitude control loop, which feeds inputs into a faster vertical velocity control loop. The result is a more interpretable, tunable, and performant control algorithm.

The control loop can be seen in Fig. 8.3 on the next page. It consumes an ESP32 vario data (vario is short for variometer, or vertical velocity), the FC IMU roll and pitch, ToF readings, barometer altitude readings, and BetaFlight’s own internally derived VARIO. We initially thought that BetaFlight’s derived VARIO would be sufficient to control on. What we found though was that it was far more variable than we expected, showing fast, almost step function like jumps in values during smooth flight. This led to poor performance in holding altitude values. We then took it upon ourselves to poll the raw sensor readings over the MSP link from the FC to the ESP32, and run our own sensor fusion and filtering logic on the sensor data.

To calculate our own ESP32 derived VARIO, we implemented a few lightweight filtering techniques to smooth out noise and add validity checks for point discontinuities in sensor data (throw out infeasibly large jumps in readings)

The first thing we tried to include was a ToF sensor to take out the barometer readings altogether, which are very unreliable near the ground, whereas the ToF is extremely reliable at low altitudes. Again, we will spend little time here because the ToF ended up not being very useful, but the math looked like as follows. At low altitudes the derived VARIO trusted the ToF heavily:

```
float tofBlendWeight(float altitudeM, bool valid) {
    if (!valid) return 0.0f;
    if (altitudeM <= TOF_BLEND_FULL_M) return 1.0f;
    if (altitudeM >= TOF_BLEND_ZERO_M) return 0.0f;
    return (TOF_BLEND_ZERO_M - altitudeM) / (TOF_BLEND_ZERO_M - TOF_BLEND_FULL_M);
}
```

Where:

$$\text{TOF_BLEND_FULL_M} = 1.60$$

$$\text{TOF_BLEND_ZERO_M} = 2.20$$

$$\text{TOF_VALID_MAX_M} = 2.50$$

So essentially:

$$w_{ToF}(z) = \begin{cases} 1, & z \leq 1.60 \\ \frac{2.20 - z}{2.20 - 1.60}, & 1.60 < z < 2.20 \\ 0, & z \geq 2.20 \end{cases} \quad (1)$$

Where z is the vertical altitude measured by the ToF sensor. Less than 1.60 meters of altitude we trust the ToF



entirely, with a weighting of one in the blend with the barometer. In between 1.60 and 2.20 meters, we will have an updated weighted blend based on Z, then if z values start to exceed 2.20 m, our ToF sensor data will start to be unreliable, and we can safely assume zero validity after 2.20 meters and go back to barometer readings only.

Numerically:

Table 8.4 ToF sensor blend confidence by altitude.

ToF altitude	Reported confidence
1.60 m or below	100%
1.70 m	83.3%
1.72 m	80%
1.80 m	66.7%
2.00 m	33.3%
2.05 m	25%
2.20 m or above	0%

The ToF altitude reading use in the ESP32 derived vario has already been corrected for drone frame tilt, since the ToF is mounted on the underside of the drone, it is subject to tilt of the drone frame in its altitude estimation.

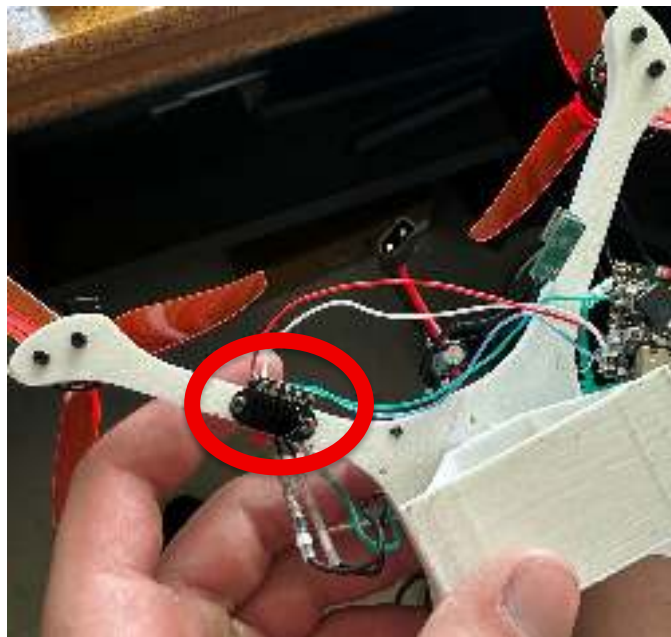


Figure 8.4. Time of Flight Sensor Mount Position.

Because the ESP32 can poll the IMU readings from the FC, we can provide a crude correction for readings based on equation 2. Where ϕ and θ are roll and pitch angles respectively.

$$z_{\text{ToF}} = r_{\text{ToF}} \cos \phi \cos \theta \quad (2)$$

To calculate the actual derived VARIO on the ESP32, a simple finite difference approximation is used for small time steps. The raw derived velocity is the altitude change divided by elapsed time:

$$v_{\text{raw},k} = \frac{z_k - z_b}{t_k - t_b}, \quad (3)$$

where:

z_b is the altitude at the previous derivative update.

t_b is the time of that update.

$t_k - t_b$ is measured rather than assumed.

Then the raw derivative is smoothed using an exponential moving average:

$$\begin{aligned} \alpha_{\text{ToF}} &= 0.65 \\ \hat{v}_k &= (1 - \alpha) \hat{v}_{k-1} + \alpha v_{\text{raw},k} \end{aligned} \quad (4)$$

For corrected barometer altitude:

$$\alpha_{\text{baro}} = 0.45$$

so

$$\hat{v}_{\text{baro},k} = 0.55 \hat{v}_{\text{baro},k-1} + 0.45 v_{\text{raw},k} \quad (5)$$

It updates only when

$$t_k - t_b \geq 180 \text{ ms}$$

Between derivative updates, the previous result is retained:

$$v_d(t) = \hat{v}_k, \quad t_k \leq t < t_{k+1}.$$

Therefore, the complete derived-vario operation is:

$$\boxed{v_{d,k} = (1 - \alpha_s) v_{d,k-1} + \alpha_s \frac{z_{s,k} - z_{s,b}}{t_k - t_b}} \quad (6)$$

where source s is selected as

$$s = \begin{cases} \text{ToF}, & \text{fresh and confidence} \geq 80\% \\ \text{barometer}, & \text{otherwise.} \end{cases}$$

This derived value is only a measurement supplied to the Kalman filter. It is not yet the final velocity used by the controller. Speaking of the Kalman filter, this is probably the most integral part of the control loop. It takes all of the velocity and altitude sources and fuses them into one smooth, estimated state vector to be supplied to our cascaded controller. Since no sensor on its own is reliable enough to be used solely for the controller, we rely on the KF to smooth and correct our state estimate from all sources.

We chose to use a linear Kalman filter, as is likely a good approximation of the dynamics of our sensors and vehicle. There is a case to be made for extending this to an extended Kalman filter for non-linear approximation of the dynamics, but the linear case was shown to be sufficient for our drone.

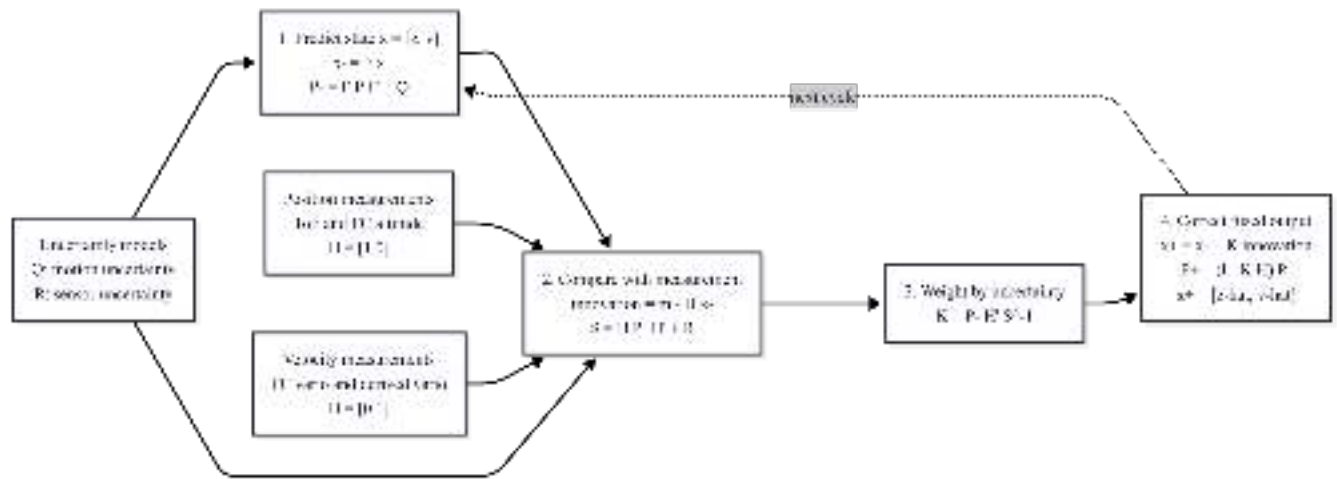


Figure 8.4. 2D Linear Kalman Filter Block Diagram.

In the case of the altitude controller, our control loop requires just a two-dimensional state vector estimate from the KF, vertical position and vertical velocity:

$$\mathbf{x}_k = \begin{bmatrix} z_k \\ v_k \end{bmatrix} \quad (7)$$

where:

z_k is vertical position in meters.

v_k is vertical velocity in meters per second.

Positive velocity represents upward motion.

I realize that in the Mechanical Engineering undergrad program we don't come across sensor fusion or Kalman filtering unless you take the Unmanned Vehicle Systems courses. For that reason, we will provide a brief overview, but full derivation and detail is out of the scope of this guide. I encourage you to go read more on state estimators. They are very useful when trying to get mechanical systems to do what you want.

A Kalman filter represents several related values using vectors and matrices. The state vector contains the quantities being estimated:

$$\mathbf{x} = \begin{bmatrix} z \\ v_z \end{bmatrix} \quad (8)$$

The covariance matrix describes the filter's uncertainty:

$$\mathbf{P} = \begin{bmatrix} \sigma_z^2 & \sigma_{zv} \\ \sigma_{zv} & \sigma_v^2 \end{bmatrix} \quad (9)$$

The diagonal terms are variances, representing uncertainty in altitude and velocity. The off-diagonal terms are covariances, representing how errors in altitude and velocity are related.

Other important matrices are:

F: predicts how the state changes over time.

Q: represents uncertainty in the motion model, such as unknown acceleration.

H: maps the state to the quantity measured by a sensor.

R: represents sensor measurement uncertainty.

K: the Kalman gain, calculated from **P**, **H**, and **R**.

A smaller measurement variance in **R** means the filter trusts that sensor more. A larger covariance in **P** means the filter is less confident in its current estimate. The Kalman gain balances these uncertainties when combining the prediction with each measurement.

Between measurements, the filter assumes approximately constant vertical velocity:

$$\mathbf{x}_{k|k-1} = \mathbf{F}_k \mathbf{x}_{k-1|k-1} \quad (10)$$

With

$$\mathbf{F}_k = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \quad (11)$$

From knowing this, the following equations is now represented in equation 12 and equation 13.

$$\hat{\mathbf{z}}_k^- = \hat{\mathbf{z}}_{k-1} + \hat{v}_{k-1} \Delta t \quad (12)$$

$$\hat{v}_k^- = \hat{v}_{k-1} \quad (13)$$

The superscript $(-)$ denotes the predicted state before incorporating new measurements.

The filter does not currently use measured acceleration as a control input. Instead, unmeasured acceleration is represented as process noise:

$$\mathbf{Q}_k = q_a \begin{bmatrix} \frac{\Delta t^4}{4} & \frac{\Delta t^3}{2} \\ \frac{\Delta t^3}{2} & \Delta t^2 \end{bmatrix} \quad (14)$$

The covariance prediction is

$$\mathbf{P}_k^- = \mathbf{F}_k \mathbf{P}_{k-1} \mathbf{F}_k^\top + \mathbf{Q}_k \quad (15)$$

Increasing q_a allows the filter to respond more quickly to unexpected acceleration, but it also makes the estimate more sensitive to noisy measurements.

There are four input sources for the KF implementation:

Table 8.3. State Measurement KF Input Sources.

Measurement	Observed state
Corrected barometer altitude	Position

Valid ToF altitude	Position
Betaflight vario	Velocity
ESP32-derived vario	Velocity

Position measurements use

$$H_z = [1 \quad 0]$$

while velocity measurements use

$$H_v = [0 \quad 1]$$

Each available measurement is applied sequentially.

Measurement Update

For a scalar measurement y_i , the innovation is

$$\tilde{y}_i = y_i - H_i x_i^- \quad (16)$$

The innovation covariance is

$$S_i = H_i P_i^- H_i^T + R_i. \quad (17)$$

The Kalman gain is

$$K_i = P_i^- H_i^T S_i^{-1}. \quad (18)$$

The state and covariance are then corrected:

$$x_i^+ = x_i^- + K_i \tilde{y}_i \quad (19)$$

$$P_i^+ = (I - K_i H_i) P_i^- \quad (20)$$

For a position measurement, the update can correct velocity as well as altitude because the altitude and velocity uncertainties become correlated during prediction. If measured altitude repeatedly exceeds predicted altitude, the filter interprets that as evidence that upward velocity was underestimated.

Measurement confidence is represented by the measurement-noise variance R . A smaller value gives a measurement greater influence.

The current values are:

Table 8.4. Chosen Variance for State Measurement Sources.

Measurement	Variance R
ToF altitude	0.02 m ²
Corrected barometer altitude	0.60 m ²
Betaflight vario	0.30 (m/s) ²
ESP32-derived vario	0.60 (m/s) ²

The chosen noise-variances are somewhat arbitrary in our case, largely visually interpreted from looking at sensor and source readings from manual flight tests.

After all available updates, the filter produces

$$\hat{\mathbf{x}}_k = \begin{bmatrix} \hat{z}_k \\ \hat{v}_k \end{bmatrix} \quad (21)$$

The estimated altitude $\hat{z}_k$ is supplied to the outer altitude-control loop. The estimated velocity $\hat{v}_k$ is supplied directly to the inner vertical-speed loop.

The cascaded controller uses two nested feedback loops.

1. The outer altitude loop that finds the current altitude error from the commanded setpoint and $\hat{z}_k$, then outputs a desired vertical velocity target

The inner velocity control loop that finds the current velocity error from the desired velocity target input from the outer loop and $\hat{v}_k$ input from the KF state estimation

Outer Altitude Loop

The sensor filtering and smoothing, MSP communication requests, and physical propulsion response all add delay to the update of the drone state in the control loop. To account for this, we implemented a small look ahead estimation.

The velocity used in this prediction is first bounded:

$$v_L = \text{sat}(\hat{v}, -v_{L,\max}, v_{L,\max}) \quad (22)$$

The predicted altitude is then seen in equation 23.

$$\boxed{z_p = \hat{z} + T_L v_L} \quad (23)$$

where T_L is the lookahead time.

Bounding v_L prevents stray velocity spikes from producing excessive predicted altitude correction. The implementation on our drone uses:

$$T_L = 0.30 \text{ s}, \quad v_{L,\max} = 1.0 \text{ m/s}$$

The altitude error is calculated against predicted altitude rather than the current estimate:

$$\boxed{e_z = z_r - z_p} \quad (24)$$

A positive error indicates that the aircraft is below the target, while a negative error indicates that it is above the target. We noticed that the drone is still jittery near target altitude, as a result, we can clamp the permitted vertical speed if we are under a certain altitude error threshold. The scheduling factor can be seen in equation 25.

$$g(e_z) = \begin{cases} g_{\min} + (1 - g_{\min}) \frac{|e_z|}{e_{\text{near}}}, & |e_z| < e_{\text{near}} \\ 1, & |e_z| \geq e_{\text{near}} \end{cases} \quad (25)$$

With current implementation values being:

$$e_{\text{near}} = 0.5 \text{ m} \quad g_{\text{min}} = 0.35$$

The outer-loop output is

$$v_r = \text{sat}(K_z e_z, -g(e_z)v_{\text{down,max}}, g(e_z)v_{\text{up,max}}) \quad (26)$$

For mission altitude hold, the current limits are:

$$v_{\text{up,max}} = 1.2 \text{ m/s} \quad v_{\text{down,max}} = 0.8 \text{ m/s}$$

Inner Velocity Loop

The inner velocity loop compares the desired vertical velocity with the KF velocity estimate:

$$e_v = v_r - \hat{v} \quad (27)$$

We consistently noticed that throttle reducing corrections had a much larger influence over the drone than throttle increasing corrections, likely due to gravity, so we assigned asymmetric proportional gains to each of the two individual cases:

$$K_v^{\text{eff}} = \begin{cases} K_v k_{\text{down}}, & e_v < 0 \\ K_v, & e_v \geq 0 \end{cases} \quad (28)$$

The current downward gain scale is

$$k_{\text{down}} = 0.55.$$

Of course, any system utilizing PID controller gains is subject to integral error wind-up risk. We accounted for that using the following:

The candidate velocity-error integral solved using equation 29. With the candidate throttle seen in equation 30.

$$I_v^c(k) = \text{sat}(I_v(k-1) + e_v(k)\Delta t, -I_{\text{max}}, I_{\text{max}}). \quad (29)$$

$$u_c = u_h + K_v^{\text{eff}} e_v + K_i I_v^c. \quad (30)$$

The candidate integral is accepted only when it does not drive the throttle farther into saturation:

$$I_v(k) = \begin{cases} I_v(k-1), & u_c > u_{\text{max}} \text{ and } e_v > 0, \\ I_v(k-1), & u_c < u_{\text{min}} \text{ and } e_v < 0, \\ I_v^c(k), & \text{otherwise.} \end{cases} \quad (31)$$

This conditional integration prevents windup when the requested throttle cannot be produced.

The final unconstrained command is

$$u^* = u_h + K_v^{\text{eff}} e_v + K_i I_v \quad (32)$$

We were able to get our altitude hold performance to perform well enough to set $K_i = 0$. We recommend that integral action only be introduced if stable proportional control shows a consistent bias.

Throttle Authority

The controller applies mode-dependent throttle limits:

$$u_{\min} = \max(u_{\text{floor}}, u_h - \Delta u_{\text{down}}) \quad (33)$$

$$u_{\max} = u_h + \Delta u_{\text{up}} \quad (34)$$

For mission hold, the current parameters are

$$\begin{aligned} u_{\text{floor}} &= 1050 \mu\text{s} \\ \Delta u_{\text{down}} &= \Delta u_{\text{up}} = 300 \mu\text{s} \end{aligned}$$

With a hover throttle of $1400 \mu\text{s}$, the normal mission limits are therefore

$$u_{\min} = 1100 \mu\text{s} \quad u_{\max} = 1700 \mu\text{s}$$

The amplitude-limited command is

$$\bar{u}_k = \text{sat}(u^*, u_{\min}, u_{\max}). \quad (35)$$

These limits preserve attitude-control authority while preventing the altitude controller from requesting excessive throttle.

Throttle Slew Limiting

The throttle command is also limited relative to the preceding command:

$$u_{\text{slew},\min} = \max(u_{\min}, u_{k-1} - \Delta u_{\downarrow}) \quad (36)$$

$$u_{\text{slew},\max} = \min(u_{\max}, u_{k-1} + \Delta u_{\uparrow}) \quad (37)$$

The final command is

$$u_k = \text{sat}(\bar{u}_k, u_{\text{slew},\min}, u_{\text{slew},\max}) \quad (38)$$

Current per-update limits are

$$\Delta u_{\downarrow} = 25 \mu\text{s} \quad \Delta u_{\uparrow} = 100 \mu\text{s}$$

At the 20 ms controller period, these correspond to approximately

$$1250 \mu\text{s/s}$$

for throttle reduction and

$$5000 \mu\text{s/s}$$

for throttle increase.

The aircraft can therefore recover quickly from a descent, while throttle removal is slower to reduce abrupt drops and ground impacts.

Sprint to Cascaded Transition Handling

During the mission, altitude control begins when fused altitude reaches `SPRINT_CUTOFF_M`, currently 15.8 m.

At the transition, the velocity integrator is cleared. The current altitude initializes the controller. The previous controller output is set to the current sprint throttle. The mission altitude setpoint changes to

TARGET_ALT_M, currently 18.3 m. The slew limiter gradually reduces throttle from the sprint value toward the closed-loop command.

This prevents an immediate throttle step when control transfers from the open-loop sprint to the cascaded controller. The early sprint cutoff and lookahead prediction account for the aircraft's remaining upward momentum.

The controller is constantly evaluating and verifying that altitude estimates are finite, and vertical velocity is fresh and physically plausible. If the estimate from the KF becomes briefly invalid:

- The previous bounded throttle is retained.
- The integral state is cleared.
- A grace period allows the estimator to recover.

If invalid state estimates from the KF persist for longer than the allowed grace period, then the state will either transition to powered landing if the mission type is powered landing type, or the controller will disarm the motors if it is the competition mission type (autorotor cut).

Web UI

A major goal for this project was to have the ability to launch the drone from anyone's phone. That was the motivation behind choosing an ESP32-S3 altitude controller board. Easily the most well supported Bluetooth capable consumer dev board on the market. To have the web UI accessible from anywhere, we knew that the interface would have to run in the browser on a publicly available URL. To achieve this we developed the UI using basic HTML, CSS, and JS to maximize web compatibility.

The ESP32 advertises as "Quad-Tuner" during Bluetooth discovery when the browser is searching for a device. Once connected, the interface initializes and shows two tabs: Flight and Params.

The Flight tab provides the BLE connection controls, mission timeline, current state, elapsed time, throttle output, altitude and vertical-speed telemetry, sensor status, flight-controller diagnostics, and mission controls. From this tab, the operator can select Angle or Acro mode, choose between Powered Landing and Autorotor Cut, run Hover and Alt Hold tests, start the complete mission, request a landing, or immediately kill the motors. A simplified Judge View shows only the flight phase, armed state, elapsed time, and throttle.

The Params tab contains the adjustable mission and altitude-controller values. Moving a slider writes the new value to the ESP32 after a short debounce period. The operator can also synchronize the interface with the ESP32, save or apply a browser preset, enable simulated-altitude Bench Mode, and download the most recent flight log. BLE values are stored in volatile memory and return to their firmware defaults after reboot; browser presets persist separately in browser local storage.

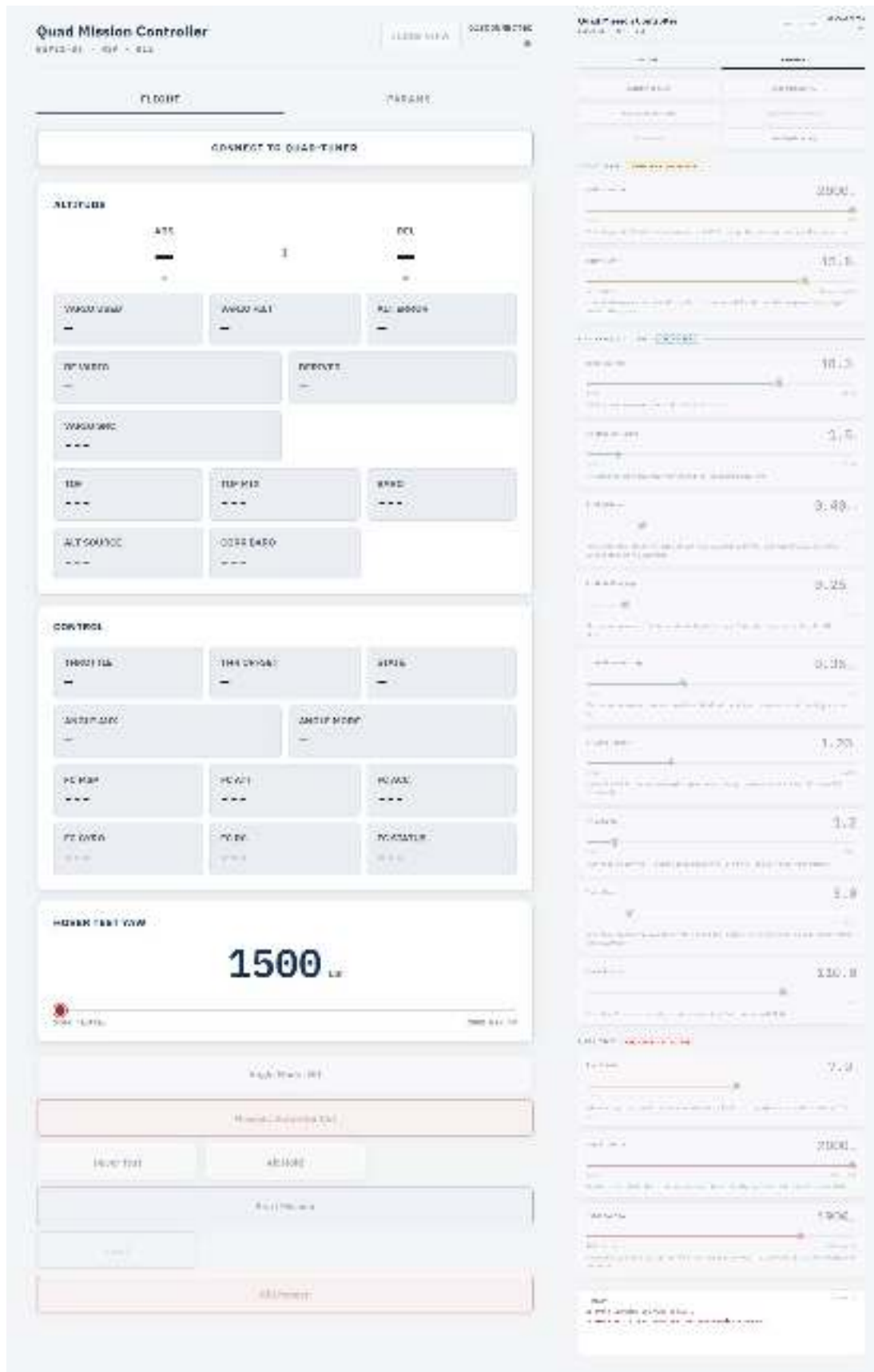


Figure 8.5. Web UI Flight and Params Tab.

Fig. 8.6 shows screenshots of the full-page views of the Flight tab and Params tab. The UI colors saturate when the ESP32 connects to the device running the web page. It is worth noting that for iOS devices, the Bluefy browser app is required to use the integrated browser Bluetooth connection.

Table 8.5. Full Tunable Bluetooth Parameter List.

UI parameter	Firmware variable	Range / increment	Default	Purpose
Hover Throttle	HOVER_THROTTLE	1000–1600 μ s / 5 μ s	1400 μ s	Baseline throttle required to approximately maintain altitude
Hover Test Yaw	HOVER_TEST_YAW	1500–2000 μ s / 10 μ s	1500 μ s	Clockwise yaw command used during Hover Test
Sprint Throttle	SPRINT_THROTTLE	1300–2000 μ s / 10 μ s	2000 μ s	Open-loop throttle used during the initial climb
Sprint Cutoff	SPRINT_CUTOFF_M	5.0–18.2 m / 0.1 m	15.8 m	Altitude where Sprint transfers to closed-loop altitude control
Target Altitude	TARGET_ALT_M	1.0–25.0 m / 0.1 m	18.3 m	Target altitude for the mission Hold phase
Alt Hold Test Target	ALT_HOLD_TARGET_M	0.5–10.0 m / 0.1 m	1.5 m	Target altitude for the standalone Alt Hold test
Alt Hold Ramp	ALT_RAMP_RATE_MPS	0.20–1.20 m/s / 0.05 m/s	0.40 m/s	Rate at which the test-mode setpoint moves toward its target
Alt Hold Climb Cap	MAX_CLIMB_MPS_TEST	0.10–1.20 m/s / 0.05 m/s	0.25 m/s	Maximum commanded climb speed in Alt Hold test mode
Alt Hold Descent Cap	MAX_DESCENT_MPS_TEST	0.10–0.80 m/s / 0.05 m/s	0.35 m/s	Maximum commanded descent speed in Alt Hold test mode
BF Vario Handoff	BF_VARIO_GROUND_EFFECT_M	0.40–3.00 m / 0.10 m	1.20 m	Height below which Betaflight vario receives reduced confidence because of ground

				effect
Alt Loop Kp	HOLD_KP	0.3–10.0 / 0.1	1.2	Converts altitude error into desired vertical speed
Speed Loop Ki	HOLD_KI	0–20.0 / 0.5	3.0	Removes steady-state vertical-speed error
Speed Loop Kp	HOLD_KD	0–150.0 / 2.5	110.0	Converts vertical-speed error into a throttle correction
Punch Start	PUNCH_START_MS	6.0–7.8 s / 0.1 s	7.0 s	Mission time at which the final burst begins
Punch Throttle	PUNCH_THROTTLE	1300–2000 μ s / 10 μ s	2000 μ s	Throttle applied during the final Punch phase
Punch CW Yaw	PUNCH_YAW	1500–2000 μ s / 10 μ s	1900 μ s	Clockwise yaw command during the final kick and Punch phase
Bench Mode	BENCH_MODE_ENABLED	Off or On	Off	Replaces sensor altitude with a simulation for desk testing
Angle Mode	ANGLE_MODE_ENABLED	Acro or Angle	Angle	Controls the Betaflight AUX2 flight-mode channel
Mission Type	MISSION_TYPE	Powered Land or Autorotor Cut	Autorotor Cut	Selects the mission-ending behavior

9. TESTING, TROUBLESHOOTING, AND ITERATION

9.1 Test Philosophy

The team used a staged test progression, so failure of one subsystem did not endanger the complete aircraft. Each stage had to be passed before moving to a higher energy test.

9.2 Recommended Test Progression

Table 9.1. Progressive test sequence and acceptance criteria.

Stage	Purpose
1. Documentation and visual inspection	Confirm part revision, wiring, soldering, fasteners, blade condition, and clearances.
2. Electrical continuity and voltage	Check for shorts, ESC battery voltage, FC regulated voltage, and connector orientation.
3. Firmware and sensor validation	Connect through USB, verify exact target, gyro, accelerometer, barometer, and board orientation.
4. Receiver and failsafe	Verify channels, endpoints, mode switches, link loss, and disarm behavior.
5. Motor test without propellers	Verify output enable, motor number, direction, ESC communication, and smooth response.
6. Rotor free spin and balance	Measure static balance, alignment, bearing friction, and clearance.
7. Low altitude powered test	Check stability, vibration, thrust margin, battery sag, and pilot procedure.
8. Controlled autorotation/drop test	Observe spin up, descent behavior, landing, and structural loads.
9. Full mission rehearsal	Combine ascent, cutoff, descent, timing, recovery, and team roles.

9.3 Electrical and Software Troubleshooting

During the competition, a wire came loose and had to immediately be resoldered. Future work by making a custom PCB would solve this issue of loose wiring. Bluetooth was acting rather slow as well, suspect of this happening is from IOS browsers.

9.4 Mechanical Troubleshooting

Fixing the ASA aero wings to the auto-rotor spars proved to be more difficult than anticipated on competition day. During the first timed flight, one of the end halves of a rotor wing disassembled from the main body. This was the first ever documented failure of the mating between a printed wing and rotor spar. Looking back on the competition retrospectively, it was likely that the prolonged heat exposure prior to flying caused the ASA Aero filament to creep around the Onyx carbon fiber spar, which lowered the interference fit friction, causing the wing to slip off of the spar during high RPM flight regime.

This situation was unique to competition, as all of our testing was conducted immediately after leaving an airconditioned building. Come competition day again, the team would recommend gluing the wing pieces to the spars themselves prior to flying. There is a trade off to be made here, as the fragile wings will likely break upon impact every flight. There was concern with doing so during competition, because that would require replacing the full spar and wing assembly in the rotor every flight. This is likely a valid tradeoff though, as the competition is only a maximum of three flights, and performance is the ultimate priority.

9.5 Data Collection

Each meaningful test recorded the aircraft configuration, mass, center of gravity, battery voltage, firmware, motor/propeller combination, rotor revision, release height, powered duration, total flight time, observed rotor behavior, and landing condition. Configuration control is essential because a result cannot be reproduced if the exact hardware and software revision is unknown.

10. RESULTS AND COMPETITION PERFORMANCE

10.1 Design Phase Predictions

The design phase analysis predicted that the propulsion system could meet the 60ft ascent requirement within the eight second powered window. The rotor model predicted sustained RPM during descent, a lower sink rate than a drag only fall, and approximately 13s of descent from the modeled height. These predictions established target behavior for physical testing.

10.2 Observed Physical Performance

The final aircraft successfully integrated the powered ascent system and autorotation descent device. Physical testing and competition operation demonstrated that the rotor could produce a useful autorotating descent after the powered phase. The aircraft completed a valid competition attempt and remained sufficiently reliable for the team to earn first place in the Rotary Wing category.

10.3 Competition Result

Table 10.1. Final competition result.

Result Item	Outcome
Competition	University of Texas at Arlington 3D Printed Aircraft Competition (3DPAC)
Category	Rotary Wing
Placement	1st place
Winning attempt	1 st attempt
Total flight/descent time	15.3 s

10.4 Comparison with the Numerical Model

The numerical model developed during the design phase predicted that the B.A.T. Wing would sustain autorotation during descent, reduce its sink rate through combined rotor lift and aerodynamic drag, and achieve a flight duration of approximately 13 seconds. During final testing and competition, the aircraft achieved a flight time of approximately 15.3 seconds, exceeding the numerical prediction by approximately 17.7%. The experimental results supported the primary behavior predicted by the model.

After the powered ascent, the rotor successfully entered autorotation and continued spinning during descent, producing aerodynamic braking and reducing the aircraft's descent rate. The longer measured flight time indicates that the physical aircraft performed better than the original numerical estimate. The difference between the predicted and measured results can be attributed to factors that were simplified or estimated within the numerical model, including the final aircraft mass, atmospheric conditions, rotor friction, blade manufacturing tolerances, actual release altitude, and aerodynamic interactions between the rotor and airframe.

Additionally, the numerical model was primarily intended to guide rotor geometry selection and compare design changes rather than provide an exact prediction of competition performance. Overall, the measured 15.3 second flight time compared with the predicted 13 second result provides strong experimental support for the numerical design approach. The approximately 17.7% improvement over the predicted value, along with the successful autorotation observed during flight, indicates that the optimized rotor design performed at least as well as expected under the final competition conditions.

10.5 Reliability and Repeatability

Competition performance required more than one strong design point. The team needed working electronics, reliable motor cutoff, rotor freedom, structural integrity, charged batteries, spare parts, tools, and clear operating roles. The first place result indicates that the final system was sufficiently reliable to produce the best scored result in its category.

11. SUMMARY AND CONCLUSIONS

The B.A.T. Wing project completed a full engineering development cycle: requirements definition, concept generation, weighted decision analysis, aerodynamic theory, numerical optimization, CAD, structural mass reduction, multi material additive manufacturing, electronics integration, flight control configuration, progressive testing, troubleshooting, and competition operation.

The weighted decision matrix correctly favored autorotation concept over higher risk glider alternatives. The AeroSandbox/NeuralFoil/CasADi workflow produced a NACA 4512 rotor geometry and predicted useful spin up and descent behavior. Additive manufacturing allowed the team to implement lightweight blade shells, structural spars, and repairable airframe components. Betaflight reduced pilot workload by automatically stabilizing the multirotor while the pilot retained high level control of the mission.

The aircraft earned first place in the 3DPAC Rotary Wing category and became the first successful competition demonstration of an autorotation based drag device. These results support the conclusion that autorotation was not only theoretically relevant but practically competitive.

12. FUTURE WORK

12.1 Electronic Improvements

For future work on electronics, making a custom PCB would be recommended for improving the electronics display and sufficient housing requirements. However, depending on the size of the PCB, the housing bay may have to be given a large area for the mounting.

12.2 Airframe and Manufacturing Improvements

For future Manufacturing improvements, finding a bigger printing bed would provide greater surface area for the wings. With more surface area available, the autorotation would be able to carry more, however the bearing would need greater torque for initial spinning.

12.3 Control and Autonomy Improvements

For future work that could be implemented into the drone. Placing another motor under the on the way bearing that could be used to provide torque for initial spinning. With this initial spin, the wings would be able to get a better start before “grabbing” the autorotation. However, finding out how small the motor would need to be researched based on how much torque it can provide. Finding this would prove if adding the necessary weight would be worth the expense.

13. ACKNOWLEDGEMENTS

The B.A.T. Wing team thanks Dr. Robert Taylor and Dr. Thomas Allsup for their guidance, technical feedback, and support throughout the project. The team also thanks Steven McDermott for providing access to laboratory space used for assembly, soldering, electronics troubleshooting, bench work, and competition preparation. That access was important to the team's ability to iterate and prepare a reliable aircraft.

The team acknowledges the University of Texas at Arlington Mechanical and Aerospace Engineering Department for organizing and supporting the 3D Printed Aircraft Competition and for providing the educational, software, and manufacturing resources used during the project.

14. REFERENCES

- [1] University of Texas at Arlington, “10th Annual 3D Printed Aircraft Competition Rules,” Mechanical and Aerospace Engineering Department, rules dated July 24, 2025, 2026. Accessed Aug. 6, 2026.
- [2] H. Glauert, “Airplane Propellers,” in *Aerodynamic Theory*, Vol. IV, W. F. Durand, Ed., Berlin, Germany: Springer, 1935.
- [3] P. Sharpe and R. J. Hansman, “NeuralFoil: An Airfoil Aerodynamics Analysis Tool Using Physics-Informed Machine Learning,” 2025.
- [4] M. Drela, “XFOIL: An Analysis and Design System for Low-Reynolds-Number Airfoils,” in *Low Reynolds Number Aerodynamics*, Lecture Notes in Engineering, Vol. 54, Springer, 1989.
- [5] L. Prandtl, “Applications of Modern Hydrodynamics to Aeronautics,” National Advisory Committee for Aeronautics, NACA Report No. 116, 1921.
- [6] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, “CasADi: A Software Framework for Nonlinear Optimization and Optimal Control,” *Mathematical Programming Computation*, Vol. 11, pp. 1–36, 2019.
- [7] A. Wächter and L. T. Biegler, “On the Implementation of an Interior-Point Filter Line-Search Algorithm for Large-Scale Nonlinear Programming,” *Mathematical Programming*, Vol. 106, pp. 25–57, 2006.
- [8] P. Sharpe, *AeroSandbox Documentation*, AeroSandbox Version 4.2.6, 2026. Accessed Aug. 6, 2026.
- [9] Betaflight Project, “Betaflight 4.5 Release Notes and Documentation,” Betaflight. Accessed Aug. 6, 2026.
- [10] Betaflight Project, “Betaflight Firmware 4.5.2,” GitHub release documentation. Accessed Aug. 6, 2026.
- [11] Altair Engineering Inc., *Altair Inspire User Guide*, Altair Inspire, 2025. Accessed Aug. 6, 2026.
- [12] Recursion Labs, “1404 Motor Performance Testing,” experimental motor and propeller performance testing, 2021.
- [13] Recursion Labs, “3-in. and 3.5-in. Motor/Propeller Performance Testing,” experimental propulsion data used for preliminary propulsion sizing, 2022.
- [14] Markforged Inc., *Onyx Composite Material Data*, Markforged material documentation, accessed Aug. 6, 2026.
- [15] Bambu Lab, *ASA Aero Technical Data Sheet*, Bambu Lab material documentation, accessed Aug. 6, 2026.
- [16] Batwing Corporation, *B.A.T. Wing: 3DPAC Rotary Wing Autorotation Aircraft*, Senior Design I Final Technical Report, University of Texas at Arlington, May 1, 2026.
- [17] Batwing Corporation, *B.A.T. Wing Senior Design II Proposal: Build, Test, and Validate a 3DPAC Rotary-Wing Autorotation Aircraft*, University of Texas at Arlington, 2026.
- [18] Batwing Corporation, B.A.T. Wing CAD models, aerodynamic optimization files, flight-control configuration files, test logs, and competition records, University of Texas at Arlington, 2026.
- [19] Recursion, “Recursion Labs - Sub 250g Motor and Propeller Testing,” RC Groups, URL: <https://www.rcgroups.com/forums/showthread.php?4009681-Recursion-Labs-Sub-250g-Motor-and-Propeller-Testing> [retrieved 6 August 2026].

APPENDIX A. COMPETITION RULES, CODES, AND COMPLIANCE

Table A.1. Rule and compliance traceability.

Source / Requirement	Exact Requirement	Design / Operating Response
3DPAC powered duration	Maximum 8 continuous seconds	Use timed cutoff procedure and observer
3DPAC altitude	Below 60ft	Vertical mission and altitude awareness
3DPAC field boundary	Approximately 300 ft x 160 ft	Avoid long glide path; use spotter
3D-printing requirement	Aerodynamic/airframe surfaces printed	ASA Aero/ASA/Onyx printed components
Electrical/propulsion safety	Safe electric system	Continuity, voltage, failsafe, prop-off testing

APPENDIX B. DECISION MATRIX SCORE JUSTIFICATION

The main report presents the matrix and the final selection. This appendix records the engineering basis for the raw scores so future teams can revise the matrix if rules, resources, or technology change.

Table B.1. Raw score justification.

Criterion	Autorotation	VTOL Tail-Sitter	Glider/Quad Hybrid
Hang time	Strong passive descent and low transition loss; score 8.	Potentially efficient glide if transition succeeds; score 9.	Added wing area but higher mass and transition/control penalties; score 5.
Weight	Compact rotor and lightweight lower frame; score 8.	Large wing/tail structure; score 5.	Combined wing and quad structure; score 6.
Fabrication	Three large blades and hub are challenging but modular; score 7.	Large wing and transition structure; score 5.	Printable but many integrated parts; score 7.
Powertrain complexity	Standard quad ascent; passive descent; score 8.	Standard motors but transition/control complexity remains; score 9 for hardware simplicity in original matrix.	Powered quad plus glider integration; score 5.
Integration complexity	Separate lower frame and rotor; score 7.	Vertical-to-forward transition, control surfaces, and CG sensitivity; score 4.	Multiple flight regimes and large interfaces; score 3.
Development effort	Clear modular tests and model; score 8.	High transition-development burden; score 4.	High integration/tuning burden; score 4.

APPENDIX C. OPTIMIZATION ASSUMPTIONS

Assumption	Implementation in Model	Effect / Limitation
Vertical descent	Motion is modeled only in the vertical direction using height, downward	Does not capture sideways drift, tumbling, pitch, roll, or yaw after

	velocity, and rotor angular velocity.	power cutoff.
Passive autorotation	No motor torque or powered thrust is applied during descent. Rotor speed changes only due to aerodynamic torque and friction.	Matches the unpowered descent rule but assumes the rotor can begin spinning immediately.
Constant atmosphere	Air density and viscosity are fixed at standard sea-level values.	Reasonable for a 60 ft drop but ignores small changes in air properties.
Rigid body and rotor	Drone body and rotor blades are assumed not to deform.	Does not account for blade bending, twisting, vibration, or structural failure.
Simplified rotor mass model	Rotor mass and inertia are estimated from shell and spar geometry.	Actual 3D-printed mass may differ due to slicer settings, infill, wall count, and print quality.
Quasi-steady aerodynamics	Blade loads are calculated from instantaneous descent velocity and rotor speed.	Does not fully capture unsteady effects such as dynamic stall or transient wake development.
2D airfoil polar data	Lift and drag coefficients are interpolated from airfoil polar tables as functions of Reynolds number and angle of attack.	Does not fully capture 3D rotating-blade effects or surface roughness from printing.
Induction approximation	Axial and tangential induction factors are solved iteratively at each blade station.	Approximates rotor wake effects but does not model the full wake structure.
Tip and hub loss correction	Prandtl-style correction is applied near the blade tip and hub.	Improves finite-blade modeling but remains an approximation.
Low-Mach operation	Rotor tip Mach number is limited to 0.35.	Keeps the model within the low-speed aerodynamic regime.
No rotor-body interference	Rotor thrust and body drag are calculated separately.	Does not capture aerodynamic interference between the drone body and rotor wake.
Fixed time step simulation	Final descent simulation uses a 0.01 s time step.	Results may have small numerical dependence on time step size.
Solver convergence requirement	Final selected design is chosen from successful optimization cases.	Designs with failed solver status are treated as unreliable even if their reported fall time is higher.

APPENDIX D. OPTIMIZATION AND SOFTWARE GITHUB REPO

[RyanHailey21/autorotation_opti_v2](https://github.com/RyanHailey21/autorotation_opti_v2)

APPENDIX E. PROPULSION CALCULATIONS AND COMPONENT DATA

Table E.1. Recursion Labs empirical data for 1404 stator current, power, and efficiency

Prop (inch)	Voltages (V)	Throttle (%)	Load Current (A)	Power (W)	Pull (g)	Efficiency (g/W)	Temperature (in full throttle load 1min)
3"	11.4	10%	0.6	9.6	36	3.75	58
		20%	1.4	22.4	75	3.35	58
		30%	2.5	40.0	118	2.95	58
		40%	4.2	67.2	169	2.51	58
		50%	5.8	92.8	205	2.21	58
		60%	7.4	118.4	236	1.99	58
		70%	9.2	147.2	281	1.91	58
		80%	10.6	169.6	307	1.81	58
		90%	12.5	200.0	352	1.76	58
		100%	14.2	227.2	379	1.67	58

Table E.2. Recursion Labs empirical data for 1404 stator-propeller pairs.

Propeller	Diameter	Pitch	Blades	Thrust (g)	Current (A)
3x2x2	3 in	Low	2	220-260	8-10
3x3x3	3 in	Medium	3	280-320	12-14
3x4x3	3 in	High	3	300-350	14-16
3.5x2.8x3	3.5 in	Med-high	3	320-380	15-18
4x2.5x2	4 in	Low	2	350-420	18-22